GlobalGrid2050 Repository Analysis Report

Deep Technical Audit — Atlas V5 Architecture, System Dependencies, Content Inventory & V7 Development Pathway

Report Date: 21 April 2026
Prepared for: Ventus Ltd — Internal Development Intelligence
Scope: Full repository audit of `Ventusltd/globalgrid2050`, with primary focus on Atlas V5 and its dependency chain (50%), and secondary coverage of the full content catalogue, knowledge base, tooling ecosystem, and identified missing patterns (50%). Goal: tighten the next development steps for Atlas V7, clarify the OS IDE environment philosophy, and identify AI integration entry points.

PART ONE — ATLAS V5 AND ITS FULL DEPENDENCY CHAIN

1.1 What Atlas V5 Is

Atlas V5 (`repd_grid_atlasv5/index.html`) is the current stable production release of the GlobalGrid2050 interactive energy intelligence platform. It is described in the architecture documentation as the "current stable" version, while V6 is its direct clone — a designated experimental sandbox intended for safe iteration. The architecture decision log (`ARCHITECTURE_V2.md`) records explicitly: "V5 remains untouched — working production atlas. V6 is a clone of V5 — safe sandbox for experimentation. V7 evolves from proven V6 changes."

Structurally, V5 is a single HTML data cartridge — a thin configuration wrapper of approximately 250 lines that wires together the Ventus OS runtime. The HTML file itself contains almost no logic. Its role is declaration, not computation. It defines the full layer configuration as a JavaScript array called `ukConfig`, sets the map centre to `[-3.5, 54.0]`(approximately the geographic centre of Great Britain) at zoom level 4.2, and then hands control entirely to `window.initVentusMap()` — the entry point into `ventus-core.js`.

This separation of concerns is architecturally deliberate and represents one of the most important structural decisions in the platform's history. The HTML cartridge is a replaceable configuration skin. The engine is centralised and reused. V3, V5, and V6 all consume the same `ventus-core.js` without modification. This is a clean horizontal scaling pattern — when a new atlas region is needed (a European version, a US version), the approach is simply to create a new HTML cartridge with a new config array pointing at new GeoJSON sources. No engine changes required.

1.2 The V5 Layer Configuration — Full Dataset Declaration

V5 declares nine layer groups, totalling approximately 48 individual map layers. These groups form the complete intelligence taxonomy of the platform:

Topology (GeoJSON) — The foundational transmission grid. Seven layers: 400kV (blue, `grid_400kv.geojson`, snap-enabled, preloaded), 275kV (red, `grid_275kv.geojson`, preloaded), 220kV (orange, `grid_220kv.geojson`, preloaded), 132kV (green, `grid_132kv.geojson`, preloaded), 66kV (purple, `grid_66kv.geojson`, preloaded), 11kV UKPN distribution substations (magenta, `grid_11kv_ukpn.geojson`, minimum zoom 13.5 — only rendered when zoomed in deeply), and Substations (white, `grid_substations.geojson`, isSubs: true, preloaded). The `isSubs: true` flag is a sentinel that instructs the engine to store the substation dataset globally for runtime topology snapping — a critical dependency for the grid line visual accuracy.

Assets (GeoJSON) — Six layers covering generation and demand assets: Nuclear power plants (lime green, filtered from `power_plants.geojson`), Gas and other generation (red-orange, same source with inverted filter), Industrial offtakers (orange, `industrial_offtakers.geojson`), Data Centres (cyan, `datacentres.geojson`), Airports (magenta, `airports.geojson`), and Railway stations (gold, `railways.geojson`). These are all preloaded — they appear when the user activates them immediately without network delay.

Supermarkets (GeoJSON) — Eleven retail chains, each with brand colours: Tesco, Sainsbury's, Asda, Morrisons, Aldi, Lidl, Waitrose, M&S Food, Co-op, Costco, Booths. These are not preloaded — they are hydrated on demand. The supermarket category is a deliberate Ventus intelligence addition: large format retail units are high-consumption sites, typically served by 11kV–33kV infrastructure, and represent compelling EV charging, rooftop solar, and BESS co-location opportunities. Their inclusion is not trivial decoration; it encodes Ventus's practical site development intuition.

Transit (GeoJSON) — Eight layers: Elizabeth Line, London Underground, DLR (filtered from `uk_metros_trams.geojson`by operator string), UK Metro networks (Tyne & Wear, Metrolink, Supertram, Nottingham, Edinburgh, Midland Metro), Trams & Light Rail, HS2 infrastructure, Eurostar route (line type), and Eurostar stations. Transit layers represent traction power loads — major grid demand nodes that constrain available capacity near railway corridors.

Leisure & Sports — One layer: Major Stadiums (`stadiums.geojson`). Stadiums are large episodic load events, relevant for grid constraint mapping and EV charging hub viability analysis.

Global Data Backbone — One layer: Subsea Data Cables (`subsea_data_cables.geojson`). This begins the platform's expansion beyond the UK to global infrastructure intelligence.

Global Supply Chain & Chokepoints — Three layers: Major ports (deep water/ferry, radius interpolated by area_ha), Inland/Minor Harbours, and Global Hydrocarbons (oil, gas, LNG infrastructure, radius interpolated by area). This category represents strategic global constraint data — essential for energy transition supply chain analysis.

Highway Infrastructure — One layer: Motorway Services (`motorway_services.geojson`). These are tier-one EV charging locations and are increasingly being equipped with grid-scale battery storage.

EV Infrastructure — One layer: EV Rapid chargers at 100kW or above (`ev_chargers.geojson`). This threshold was deliberate — rapid DC chargers at this power level are significant grid loads and demand-side planning assets.

REPD (GeoJSON) — Twelve technology sub-layers, all drawing from a single unified source (`dist/repd_master.json`) and split by the `tech` property: Solar PV (yellow), Rooftop Solar (amber, with capacity-gradient hue mapping), Wind (cyan), Battery Storage / BESS (orange), Biomass & EfW (lime), Tidal & Wave (deep blue), Hydrogen (white), Hydro & Pumped Storage (light blue), Flywheel (pink), Advanced Conversion (orange-red), Geothermal (red), and Compressed Air Storage (soft blue). The REPD layer group is special: it loads as a single unified dataset but renders as twelve separate visual layers. When any REPD layer is activated, the engine fetches the entire dataset once, populates the search index, computes the tech distribution statistics, and marks all twelve REPD IDs as loaded simultaneously.

1.3 The Engine — `ventus-core.js` (1,170 Lines)

`ventus-core.js` is the Ventus OS engine. It is a self-contained, zero-dependency (aside from MapLibre) monolith. It is 1,170 lines of strict-mode JavaScript exposing a single public function: `window.initVentusMap()`. The architecture documents explicitly warn: "Do not move logic back into HTML data cartridges."

The engine is organised into logical sections:

Earth Model & Named Constants. WGS84 equatorial radius (6378.137 km). Maximum geodesic radius (π × R = 20,037.508 km — the equatorial half-circumference). All geometry uses Haversine logic. The comment in the code is instructive: "All calculations must use geodesic metres via Haversine logic. No shortcuts permitted." Named constants are consolidated into a single block (V5.1 improvement): `DEG_TO_RAD`, `HIT_RADIUS_VERTEX_PX` (18px), `HIT_RADIUS_EDGE_PX` (22px), `CLICK_DEBOUNCE_MS` (220ms), `HOVER_THROTTLE_MS` (100ms), `POPUP_MAX_WIDTH` (300px), `ZONE_DRAW_VERTICES` (24), `ZONE_DRAW_DEFAULT_KM` (0.337 — approximately 50 football pitches).

Config Loading. The config is deep-frozen immediately (`Object.freeze()`) after loading. An O(1) `Map` lookup (`layerConfigById`) is built once on startup to replace the repeated `flatMap().find()` pattern from earlier versions.

FetchQueue. A concurrency-limited fetch queue allows maximum 4 simultaneous network requests. Each individual fetch has a 15-second AbortController timeout. GeoJSON responses are cached in `urlCache{}` — fetched once per browser session per URL.

Topology Snapping. The `snapLines()` function aligns grid line endpoints to the nearest substation coordinate within a ~111m tolerance. This is explicitly flagged as a tech debt in the source code: "TECH DEBT: move to build pipeline to remove runtime cost entirely. Retained here temporarily to preserve physical grid truth." The tolerance check deliberately uses planar squared-distance with cosine latitude correction rather than full Haversine, since the nested loop

cost across ~5,800 substations × line endpoints at load time is measurably expensive and the error is below 0.1% at UK latitudes.

Visible Layer Cache. Two caches: `_visibleInteractiveIds`(click-sensitive) and `_visibleHoverIds` (mousemove-sensitive). These are rebuilt on every toggle to avoid querying `map.getLayoutProperty()` on every mouse event — a significant performance optimisation for high-density map interactions.

Spatial Tools. Five distinct interactive tools, all mutually exclusive (activating one deactivates all others):

- *Radius Search* — Click-to-place geodesic circle (1km–20,037km). Queries loaded REPD features and reports total MW and tech breakdown within radius.
- *Radius Area Measure* — Geodesic spherical cap area calculation using the formula `2πR²(1 − cos(r/R))`. Reports area in m², hectares, acres, km², miles², and football pitches.
- *Measure Tool* — Click-to-add multi-point polyline with double-click polygon closing. Calculates total path length (Haversine segment summation) and closed polygon area (spherical excess formula).
- *Poly Zone / Zone Draw* — Drop a 24-vertex geodesic circle on first click, then drag vertices or click edge dots to reshape. Full area and perimeter readout. Collapsible inline popup. Auto-zooms to fill 60% of viewport on placement.
- *Status Colours* — Toggles all REPD layers between tech-colour mode and planning-status colour mode (Operational: #00ff88, Under Construction: #ffcc00, Consented: #ff8800, Applied: #8888ff).

SCADA UI. The layer panel is built dynamically from the config, producing a grid layout of checkbox-toggled layer rows. Group headers, colour-coded labels, and live state indicators ([WAIT], [LOAD], [OK], [FAIL], or "[count | MWh]" for REPD). Fully mirrored in the fullscreen curtain panel.

Clock. A 1-second setInterval renders the current system time and date, plus a countdown in days to 1 January 2050 — the Net Zero target.

Fullscreen Mode. Enters browser fullscreen on the map container. A slide-down curtain panel provides the layer control UI in fullscreen. State is synchronised bidirectionally between the standard panel and the curtain.

Search. When any REPD layer is loaded, a search index is built. The search input fires on keystroke, filtering by project name (case-insensitive substring) and sorting by capacity descending, returning up to 12 results. Clicking a result calls `map.flyTo()` at zoom 12 with a 1.8-second animation, then opens an info popup after the flight completes. Large projects include links to Google News and Google Images searches.

Export. The Export CSV button produces a client-side CSV of all visible REPD features (name, tech, raw_tech, capacity_mw, status, operator, mounting, longitude, latitude), creates an Object URL blob, clicks it programmatically, and revokes the URL after 1 second.

1.4 The Styling System — `ventus.css` (158 Lines)

`ventus.css` is the global presentation layer. It enforces a strict SCADA-terminal visual identity: black backgrounds (#000, #050505, #0a0a0a), Courier New monospace typography, cyan (#00ffff) as the primary interactive accent, amber (#ff9d00) for countdown, and category-specific neon colours for layer types. It covers: the HUD header, map container, fullscreen states, radius and area popups, measure display panel, poly zone panel, search bar, search results, SCADA keys panel, status legend, scrollbars, fullscreen letterhead, focus-visible keyboard accessibility states, and responsive breakpoints at 480px and 600px viewport height.

A notable LCP (Largest Contentful Paint) optimisation was added: `content-visibility: auto` on `.disclaimer-box` — explicitly commented as "LCP fix: disclaimer box is the largest paint element — push it off the critical path."

1.5 The Data Pipeline — Python ETL Engine

The REPD pipeline (`scripts/repd_updater.py`, version 5.10) is the most mission-critical Python script in the repository. It:

1. Discovers the latest REPD CSV URL by scraping the UK Government publication page (using BeautifulSoup)
2. Checks a local manifest (`dist/manifest_v4.json`) to skip processing if the source URL is unchanged
3. Downloads the CSV with a 60-second timeout
4. Validates the schema (required columns + optional mounting column)
5. Filters rows to only viable planning statuses
6. Converts OSGB36 Easting/Northing to WGS84 longitude/latitude using `pyproj` (EPSG:27700 → EPSG:4326)
7. Validates geographic bounds (UK bounding box: lon -9.0–2.5, lat 49.0–61.0)
8. Classifies each record into 12 technology categories using substring matching (hydrogen checked before hydro to prevent collision)
9. Handles capacity unit normalisation edge cases (rooftop solar and biomass kW→MW corrections for likely encoding errors)
10. Outputs `dist/repd_master.json` as a unified GeoJSON FeatureCollection

Grid data is fetched separately via `scripts/fetch_grid_data.py` and its voltage-specific siblings, all querying the OpenStreetMap Overpass API. Rate limiting is handled with `time.sleep()` pauses between voltage fetches.

The automation layer (`.github/workflows/`) contains 18 GitHub Actions workflows — effectively a scheduled ETL and deployment pipeline that keeps the repository's data files current without manual intervention.

PART TWO — THE FULL CONTENT CATALOGUE, KNOWLEDGE BASE, AND MISSING PATTERNS

2.1 Atlas Version History and Evolution

The repository preserves the full atlas lineage: V1 (`repd_grid_atlasv3/index.html` — note the directory naming inconsistency, a missing pattern in itself), V3 (proper increment), V4 (the last version with all CSS inlined into the HTML file — visible from the 50-line `<style>` block at the top of `repd_grid_atlasv4/index.html`), V5 (externalized CSS, externalized JS engine, clean architecture), and V6 (clone of V5, experimental sandbox). The jump from V1 to V3 is a gap — either V2 was informal, or naming was inconsistent. The `repd_grid_atlasv3` directory is also called "V1" in the repository map ("UK Energy Atlas V1 (Grid Overlay)"), suggesting the folder-name-to-version relationship is not consistent. This is a missing pattern to formalise in V7.

The evolution from V4 to V5 is the critical architectural leap: the move from monolithic HTML (with all CSS and JS inline) to the split `ventus-core.js` + `ventus.css` pattern. V4 still has its full dashboard CSS inside a `<style>` tag. V3 (the earliest preserved version) also has full inline CSS. The V5 architecture represents the crossing of a threshold: the engine became reusable, the styling became reusable, and HTML files became pure config cartridges. V6 is identical to V5 currently — the sandbox exists but no experimental features have been committed to it yet.

2.2 The Knowledge Base — Technical Reference Content

The repository contains a substantial technical knowledge base, all rendered as GitHub Pages markdown content:

- `ac_cables_knowledge/` — AC cable reference
- `dc_cables_knowledge/` — DC cable reference
- `conductor_resistances/` — Resistance tables
- `definitions/` — Industry terminology
- `mv_and_hv_components/` — MV/HV equipment reference
- `solar_components/` — Solar PV component reference
- `power_systems_studies/` — Power systems studies guidance
- `sld_single_diagrams_diagrams_and_grids/` — SLD reference
- `employers_requirements/` — Employer requirements template
- `nsip_solar_farms/` — Nationally Significant Infrastructure Project solar guidance

This content represents codified Ventus domain expertise — two decades of physical engineering knowledge translated into reference documentation. It is directly referenced in the architecture: "This platform codifies 2 decades of physical engineering experience into a spatial reasoning system." However, these pages exist as static markdown files. They are not yet connected to the atlas. When a user clicks on a substation or a cable type in the map, there is no link to the relevant knowledge base article. **This is a significant missing connection pattern** — the knowledge base and the interactive atlas operate as parallel but disconnected experiences.

2.3 Tools and Calculators

Five interactive tools are deployed alongside the atlases:

`33kv_uk_dap_price_estimator/` — A 33kV UK Distribution Answers Panel price estimator. This addresses one of the most common real-world questions in MV grid connection: what does a 33kV connection cost?

`lv_ac_dc_price_estimator/` — LV AC and DC distribution cable pricing. This is directly tied to the commodity price tracking work in the repository (`copper_and_aluminium_prices_historic_trends.md`, `ons-energy-fuels-clean.csv`).

`cable_geometry/` — A cable geometry visualiser (V2). Has its own data modules: `data-core.js`, `data-lv.js`, `data-mv-hv.js`, `data-solar.js`. These are separate datasets not connected to the atlas config system. **Missing pattern:** the cable geometry tool has not been integrated into the main atlas engine.

`solar-bess-topology/` — Cable Geometry Visualiser V1 (earlier version, preserved).

`cable_selection/` — A cable selection tool with `data.json`and a `schema.md`. Again, a standalone tool with its own data, disconnected from the central config registry. **Missing pattern:** these tool-specific data files are not listed in `config/registry.yaml`.

2.4 The Configuration Registry — `config/registry.yaml`

The registry is version 4.3 with `global_compliance: "EDTF_2019_PRESUMED_OPEN"`. It catalogues all the data layers with metadata including `url`, `type` (csv, geojson, osm, api), `tags`, `refresh` schedule (monthly, manual, fortnightly), `output` path, and `script` reference. This is the single source of truth for the data pipeline.

However, the registry is incomplete. It covers the core grid and asset layers, but the supermarket layers (14 chains), the subsea data cables, global ports, global hydrocarbons, stadiums, motorway services, EV chargers, and EV data are listed only partially or not at all in the registry file that was accessible for review. **Missing pattern:** the registry is not fully synchronised with what is actually declared in the V5/V6 config. A complete registry would list every layer that appears in the atlas config, with its refresh schedule, script reference, and version tag.

2.5 The Gridbot Automation Layer — 18 GitHub Actions Workflows

The 18 workflows form what the architecture calls "Gridbot" — the automated system that maintains data truth. Notable workflows include:

- `repd_sync.yml` — REPD data sync (the most critical data pipeline trigger)
- `update-grid-data.yml` — HV transmission grid refresh from Overpass API
- `update_11kv_ukpn.yml` — 11kV distribution substations (UKPN area only)
- `fetch_elizabeth_line.yml`, `fetch_eurostar.yml` — Transit data
- `fetch_global_ports.yml`, `generate_global_hydrocarbons.yml` — Global infrastructure
- `update_ev_chargers.yml` — Open Charge Map API integration
- `update_prices.yml` — Commodity pricing
- `generate_repo_structure.yml` — Auto-generates `REPO_STRUCTURE.txt`
- `jekyll-gh-pages.yml` — Static site deployment (the final publishing step)

Missing pattern: there is no monitoring or alerting workflow. When a workflow fails (Overpass API down, Gov.uk REPD page structure changes, Open Charge Map API rate-limited), there is no automated notification or health dashboard. A `gridbot_health.yml` workflow that runs daily, checks manifest freshness, validates GeoJSON file sizes against expected minimums, and posts a status to a Slack webhook or GitHub issue would close this gap significantly.

Missing pattern: refresh schedules are declared in the registry YAML but not enforced by cron syntax in the workflows consistently. Some workflows use `workflow_dispatch` only (manual trigger), while others have scheduled crons. A full audit of refresh cadence vs declared cadence would reveal drift.

2.6 The Ventus Core V2 — Parked Architecture

`ventus-core-v2/` is a parked Phase 1 modular rewrite. It contains 52 files (23 implemented, 29 placeholders), 78 passing unit tests, and a full Redux-like StateStore, URLCache with TTL, AbortController management, CleanupRegistry, ListenerRegistry, and pure geometry functions. It was built in a 7-hour session and then deliberately parked when the architectural lesson was drawn: "Working code beats theoretical architecture. Incremental beats wholesale replacement."

This is one of the most instructive documents in the repository — a cautionary record of scope drift. The V2 blueprint was positioned as a "systems engineering correction" addressing real technical debt: memory leaks in `requestAnimationFrame`, unowned global state, lack of lifecycle management, no unit tests. Those issues are real. But the decision to park V2 and instead iterate V5→V6→V7 incrementally is sound given the working state of the production atlas.

However, the V2 foundation provides a precise roadmap of what V7 needs to incorporate over time: proper `destroy()` lifecycle, StateStore, AbortController ownership, URL cache with TTL eviction, and event-driven architecture (`map.on('layer:loaded', ...)` style API). These are not theoretical improvements — they directly address known failure modes (the 8-hour dashboard crash, unbounded memory growth in long sessions, fetch cancellation on tab close).

2.7 The Marketing and Showcase Layer

`marketing/earth.html` — A 3D WebGL Earth showcase page. `marketing/showcase.html` — "VENTUS Global Data Backbone" 3D WebGL engine showcase. These suggest a visual capabilities demonstration layer that exists separately from the functional atlas. This is a deliberate positioning asset — demonstrating what the engine can render at its most visually impressive.

Missing pattern: the marketing showcase and the functional atlas are architecturally disconnected. A unified entry point page (`index.html` at root is the only current landing page) does not clearly articulate the platform's capabilities hierarchy: from globe showcase to atlas to tools to knowledge base. The navigation architecture is not systematic.

2.8 Podcast Transcripts and Blog

`podcast_transcripts/index.md` and `blog/index.md`represent the editorial layer of the platform. The blog supports the "In support of The Future of Solar Photovoltaics podcast" credit that appears in every atlas interface. **Missing pattern:**there is no RSS feed, structured article schema,

or article-to-atlas deep-link pattern. Articles that reference specific grid assets or REPD projects cannot currently link to an atlas view centred on those coordinates. This is the key "article writing" integration gap referenced in the development brief.

2.9 Solar and Grid Dataset Atlases

Beyond the main V5 atlas, the repository deploys several specialised atlases as Markdown pages:

- `uk_offshore_wind_atlas.md` — Offshore wind
- `uk_onshore_wind_atlas.md` — Onshore wind
- `uk_operating_large_solar_farms.md` — Solar >4MWp operational
- `uk_solar_in-planning-1-50mw.md` — Solar pipeline 1–50MW
- `uk_grid-batteries.md` — BESS tracker
- `repd-uk-operational-solar.md` — Operational solar
- `solar_deployment_statistics/` — Deployment statistics

These are separate editorial/data pages, not instances of the V5 engine. They likely use either the older V1-style inline MapLibre pattern or are purely data tables. **Missing pattern:**these specialised atlases should be refactored as V5/V6-style cartridges using `ventus-core.js` so all atlas pages benefit from the same tool set (radius search, poly zone, export CSV, status colours).

PART THREE — IDENTIFIED MISSING PATTERNS AND V7 DEVELOPMENT PRIORITIES

3.1 The Topology Snap Tech Debt

The most explicit and self-documented tech debt in the entire codebase is runtime topology snapping. The comment in `ventus-core.js` at line 699 says it plainly: "TECH DEBT: move to build pipeline to remove runtime cost entirely." The `snapLines()` function runs in every user's browser on load of transmission grid layers, doing a nested O(features × substations) planar distance check. For V7, this must be a build-time operation: a Python script in `scripts/` that pre-snaps all line GeoJSON files during the `update-grid-data.yml` workflow and writes snapped output. The browser should receive already-snapped topology. This eliminates the runtime cost and the `globalSubsData` dependency chain entirely.

3.2 The URL Cache Missing TTL

`ventus-core.js` implements a simple object `urlCache{}` as a promise cache. It persists indefinitely for the lifetime of the browser tab. On a long-running dashboard session (8+ hours — the documented failure mode), this cache will never refresh stale data even if the underlying GeoJSON has been updated. The V2 architecture implemented a 5-minute TTL `URLCache`with auto-eviction and a `visibilitychange` listener to pause updates when the tab is hidden. Both these patterns need to graduate from the parked V2 into V6/V7 incrementally.

3.3 No Constraint Intelligence Layer

The architecture document (`ARCHITECTURE.md`) devotes an entire section to the Constraint Intelligence Layer — the most forward-looking part of the system vision. It defines two modes: Asset Mode (what exists, what is planned) and Constraint Mode (what limits development, what blocks routes). Currently, V5 and V6 only implement Asset Mode. The constraint layer — overhead line proximity, pipeline conflicts, corridor compression — is defined in the architecture but has zero implementation in the actual atlas. For V7, at minimum one constraint layer source must be ingested (Open Infrastructure Map data), normalised into Ventus schema with `constraint_type`, `confidence_level`, and `source_origin`tags, and rendered as a distinct visual layer category (red/magenta = hard constraint, cyan = corridor intelligence).

3.4 No Plugin or Extension Architecture

Every feature currently ships as a modification to `ventus-core.js`. There is no way to add a new tool, layer renderer, popup template, or search provider without modifying the monolith. The V2 blueprint articulated a Plugin API where plugins register tools, controls, and search providers against a narrow core API. This is the prerequisite for the "new app additions" goal in the development brief — the ability to add new capabilities to the IDE-style environment without touching the stable engine.

3.5 No Article-to-Map Deep Link Pattern

The blog and podcast transcripts have no mechanism to embed a live atlas view or link to a specific map coordinate/zoom/layer state. A V7 URL pattern like `/repd_grid_atlasv7/?lat=51.5&lon=-0.1&zoom=12&layers=solar,bess` would allow articles to link directly to pre-configured map states. This is the "article writing using ALL new AI technologies" connection — an AI assistant generating an article can include parametric map embed links as citations.

3.6 Directory Naming Inconsistency

`repd_grid_atlasv3` contains V1 functionality (per the repository map), `repd_grid_atlasv4` is V4 with inline CSS, `repd_grid_atlasv5` is V5. The numbering jumped from "V1" to "V3" in folder names. V7 should establish a clean naming convention: if the folder is `repd_grid_atlasv7`, the title should be "Atlas V7" with no ambiguity.

3.7 No Accessibility Audit

`ventus.css` includes `focus-visible` keyboard accessibility states for buttons, but there is no systematic WCAG 2.1 AA audit. The V2 blueprint listed accessibility as Phase 3 work. For a platform serving infrastructure professionals and referenced in public-facing articles, accessibility compliance is an expectation.

3.8 No Typing or JSDoc

The 1,170-line `ventus-core.js` has no JSDoc comments on functions and no TypeScript types. The V2 architecture noted TypeScript migration as Phase 4 work. For a complex spatial engine, typed interfaces for `LayerGroup`, `LayerConfig`, `RuntimeState`, and tool state objects would prevent regression during V7 development.

PART FOUR — RECOMMENDED V7 DEVELOPMENT PATHWAY

Based on the full audit, the recommended V7 development order, each step independently shippable:

Step 1 — Pre-snap build pipeline. Move `snapLines()` to `scripts/snap_topology.py`. Run it as part of `update-grid-data.yml`. Remove runtime snapping from `ventus-core.js`. Ship in V6 first, promote to V7. High-impact, no user-visible regression.

Step 2 — URL cache TTL and visibilitychange. Extract the V2 URLCache implementation and integrate it into `ventus-core.js` V6. Set a 30-minute TTL for GeoJSON data.
Add `visibilitychange` listener to pause interval timers when the tab is hidden. Address the 8-hour crash scenario.

Step 3 — URL-state permalinking. Parse `window.location.search` on init to set center, zoom, and active layers from URL parameters. Update the URL on map interaction without page reload (using `history.replaceState`). This unlocks article deep-linking.

Step 4 — First constraint layer. Add a single constraint layer to V6: the Open Infrastructure Map overhead line network, fetched via Overpass API with `power=line` plus voltage filter, normalised with constraint metadata tags, and rendered in distinct red/magenta. Wire it into the V6 config as a new group: "Constraint Intelligence (Reference)." Label as reference data only.

Step 5 — Registry synchronisation audit. Audit `config/registry.yaml` against the full V5/V6 layer config. Add all missing entries. Establish registry as the authoritative layer catalogue for Gridbot automation.

Step 6 — Gridbot health workflow. Add `gridbot_health.yml`: daily manifest freshness check, GeoJSON file size validation, workflow failure notification via GitHub issue or Slack.

Step 7 — Specialised atlas refactor. Convert `uk_offshore_wind_atlas.md`, `uk_onshore_wind_atlas.md`, and `uk_grid-batteries.md` to V5-style cartridge HTML files, consuming `ventus-core.js`. Each gets the full tool set.

Step 8 — AI integration entry points. Connect knowledge base articles to atlas via contextual popups. When a user clicks a 33kV substation, the popup can include a link to `ac_cables_knowledge/` or the `33kv_uk_dap_price_estimator`. When a user clicks a large BESS project, link to `employers_requirements_BESS.md`. This is the IDE-style "ambient knowledge" layer.

CONCLUSION

The GlobalGrid2050 repository is a technically coherent, production-grade spatial intelligence platform built on a clean separation of concerns: data pipeline (Python ETL + GitHub Actions), engine (`ventus-core.js` + `ventus.css`), and configuration cartridges (HTML atlas files). V5 is a stable, well-documented production release. V6 is correctly positioned as the experimental sandbox. The foundation for V7 is sound.

The primary technical debts are localised and self-documented: runtime topology snapping, URL cache without TTL, no constraint intelligence layer, no plugin architecture, and no deep-link pattern. The knowledge base and tools are rich but disconnected from the atlas engine. The blog and podcast layers have no structured linkage to atlas states.

V7 should not be a rewrite. The parked V2 experience teaches this directly. V7 should be the accumulated result of eight to ten discrete, independently shippable improvements applied to the V6 sandbox, each validated before promotion. The architecture's own final directive is the clearest possible roadmap instruction: "Every feature must answer: Does this improve constraint awareness? Does this preserve physical truth? Does this prevent bad design early? Does this increase decision clarity? If not, it does not belong."